Assignment2_finished

October 23, 2026

CSE 242 Assignment 2

3 Questions, 100 pts

`Your name: Nick Crispino  Student ID: 2253804`

0.1 Instruction

- Submit your assignments onto **Canvas** by the due date. Upload a zip file containing:
  (1) The saved/latest .ipynb file.
  (2) Also save your file into a pdf version, if error appears, save an html version instead (easy to grade for written questions).
  (3) All other materials to make your .ipynb file runnable.

  For assignment related questions, please reach TA or grader through EdDiscussion.
- This is an **individual** assignment. All help from others (from the web, books other than text, genAI, or people other than the TA or instructor) must be clearly acknowledged.
- Most coding parts can be finished with only 1-2 lines of codes.
- Make sure you have installed required packages: scikit-learn.
- Double click to edit each markdown cell.

0.2 Objective

- **Task 1:** Maximum likelihood estiamtion (math)
- **Task 2:** Linear Regression (with Scikit-learn)
- **Task 3:** Principle Component Analysis (with Scikit-learn)

1 Question 1 (Maximum likelihood estiamtion and Posterior, 20 pts)

Assume we have a coin that has some unknown probability h of coming up heads (and probability $1-h$ of coming up tails). If the coin is flipped five times getting three heads and two tails (**HHHTT**) then:

(a – 10pts) What is the maximum likelihood estimate for h?

(b – 10pts) Assume that (before flipping the coin) we have a prior density $p(h)$ for the various values of $h \in [0, 1]$. Give the formula for the posterior probability density $p(h \mid \mathbf{HHHTT})$ as a function of the prior $p(h)$.

For Question 1(b), the solution is acceptable if the formula contains an integral.

Solution (a):

Let $n = 5$ be the number of flips, h the probability of heads, x be the number of heads, $D =$ **HHHTT**.

$$\begin{aligned}\hat{h}_{MLE} &= \text{argmax}_h P(D \mid h) \\ &= \text{argmax}_h h^x (1-h)^{n-x} \\ &= \text{argmax}_h h^3 (1-h)^2\end{aligned}$$

Take the derivative wrt h to find the max:

$$\begin{aligned}\frac{dP(D \mid h)}{dh} &= 3h^2(1-h)^2 + h^3 * 2(1-h) * (-1) \\ &= 3h^2(1-h)^2 - (2h^3 - 2h^4) \\ &= 3h^2(1 - 2h + h^2 - 2h^3 + 2h^4 \\ &= 3h^2 - 6h^3 + 3h^4 - 2h^3 + 2h^4 \\ &= 3h^2 - 8h^3 + 5h^4 \\ &= h^2(3 - 8h + 5h^2) \\ &= h^2(5h - 3)(h - 1)\end{aligned}$$

Setting this equation equal to 0, we get the maximum likelihood estimate as $\hat{h}_{MLE} = 3/5$. We can confirm this is the maximum; the other two solutions are $h = 0, 1$, which both evaluate to 0 when plugged into the $P(D \mid h)$ equation, whereas $h = 3/5$ plugged in gets $(3/5)^3(1 - 3/5)^2 > 0$, so is the maximum.

Solution (b): The posterior is

$$P(h \mid D) = \frac{P(D \mid h)p(h)}{P(D)}$$

$$P(h \mid D) = \frac{h^3(1-h)^2 p(h)}{\int_0^1 P(D|h)p(h)dh}$$

$$P(h \mid D) = \frac{h^3(1-h)^2 p(h)}{\int_0^1 h^3(1-h)^2 p(h)dh}$$

If you are not familair with Latex, you may attach a figure/screen-shoot and display the code below.

[ ]:
```
from IPython.display import Image
# Replace the figure name
# Image(filename='sample.png')
```

2 Question 2 (Linear Regression, 40 pts)

In this question, you will be using **Scikit-learn** to empirically apply the Linear Regression model. We will adopt an advertisement dataset **Advertising.csv** from Kaggle.

2.0.1 Reading data using Pandas

[18]:
```
# Read the dataset you will be working on
# The dataframe loaded with pandas is named as data

import pandas as pd
data = pd.read_csv('Advertising.csv', index_col=0)  # Modify your data path␣
 ↪accordingly.

# Take a look at the first 5 rows
data.head()
```

[18]:

	TV	Radio	Newspaper	Sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
5	180.8	10.8	58.4	12.9

[19]:
```
data.shape
```

[19]:
```
(200, 4)
```

2.0.2 Features and responses

What are the features? - **TV:** advertising dollars spent on TV for a single product in a given market (in thousands of dollars) - **Radio:** advertising dollars spent on Radio - **Newspaper:** advertising dollars spent on Newspaper

What is the response? - **Sales:** sales of a single product in a given market (in thousands of items)

What else do we know? - There are 200 **observations**, and each observation (each row) is a single market. - **Your main task**: predict the sales of a single product in a given market. - Since the response variable is continuous, this is a **regression** problem.

2.0.3 Linear regression

Pros: fast, no tuning required, highly interpretable

Cons: adopting the linear regression is unlikely to always generate the best predictive accuracy, since Linear Regression presumes a linear relationship between the features and response. This assumption may not always make sense. (But if one can *transform* the features so that do have a linear relationship, sometimes progress can be made, for example by taking logs.)

Expression:

- y: the response
- β_0: the intercept
- β_1: the coefficient for x_1 (the first feature)
- β_i is the coefficient for x_i (the ith feature, $i \in \{1, 2, ..., n\}$)

For this regression task:

$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$

The β values ($\beta = [\beta_0, \beta_1, \beta_2, \beta_3]$) are called the **model coefficients**. These values are "learned" during the model fitting step using the "least squares" criterion. Then, we can make use of the fitted model to make predictions!

2.0.4 Prepare the dataset for training use

- Scikit-learn expects X (feature matrix) and y (response vector) to be NumPy arrays.
- Pandas is built on top of NumPy, and exposes numpy methods. Thus, X can be a pandas DataFrame, y can be a pandas Series!

[20]:
```
# Prepare the feature X
X = data[['TV', 'Radio', 'Newspaper']]

# check the type and shape of X
print(type(X))
print(X.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
(200, 3)
```

[21]:
```
# Prepare the response Y
y = data['Sales']
# check the type and shape of y
print(type(y))
print(y.shape)
```

```
<class 'pandas.core.series.Series'>
(200,)
```

2.1 Question 2.1 (Split the dataset into train and test parts, 5 pts)

For features X and response y, use **sklearn** to perform the the splitting of dataset: 80% for train data, 20% for the test data.

[22]:
```
################## Your answer for Question 2.1   ##################
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

################## Your code above ##################
```

Double check the shape of train and test data:

[23]:
```
# default split is 80% for training and 20% for testing
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(160, 3)
(160,)
(40, 3)
(40,)
```

2.2 Question 2.2 (Train a Linear regression model via Scikit-learn, 10 pts)

Assume you model is named as **model**, and you train the linear regression model to fit on the training data.

[24]:
```
################## Your answer for Question 2.2   ##################
# Name your linear regression model as 'model', so proceeding cells won't run␣
 ↪into errors.
from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(X_train, y_train)

################## Your code above ##################
```

[25]:
```
# Take a look at your model coefficients
import numpy as np
print(np.round(model.intercept_, 4))
print([np.round(val, 4) for val in model.coef_])
```

```
2.9791
[np.float64(0.0447), np.float64(0.1892), np.float64(0.0028)]
```

[26]:
```
# Match the feature names with the trained coefficients
list(zip(data.columns[:3].tolist(), model.coef_))
```

[26]:
```
[('TV', np.float64(0.044729517468716326)),
 ('Radio', np.float64(0.18919505423437652)),
 ('Newspaper', np.float64(0.0027611143413671935))]
```

2.3 Question 2.3 (Interpreting model coefficients, 10 pts)

The coefficients of each feature are printed above.

2.3.1 Your tasks:

- What is the trained linear regression model?

- **(replace β_i below with the trained coefficients, reserve four decimal places, 5 pts)**

$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$

- **(Open) question (5 pts)**: how do you interpret the coefficient of **TV** (β_1)?

2.3.2 Your solution for Question 2.3:

The trained regression model is below:

$$\text{Sales} = 2.9791 + 0.0447 \times \text{TV} + 0.1892 \times \text{Radio} + 0.0028 \times \text{Newspaper}$$

The coefficient of TV means a 1000 dollar raise in the advertising spent on TV for a single product in a given market results in an approximately 44.7 item increase in the sales of a single product in a given market.

2.3.3 Making predictions

[27]:
```
# make predictions on the testing set
y_pred = model.predict(X_test)
# Print the test accuracy score from your trained linear regression model
model.score(X_test,y_test)
```

[27]:
```
0.899438024100912
```

2.4 Model evaluation metrics for regression

Evaluation metrics for classification problems, such as **accuracy**, are not useful for regression problems. Instead, we need evaluation metrics designed for comparing continuous values. (Since predictions would hardly ever exactly match the measured quantities.)

Mean Squared Error (MSE) is the mean of the squared errors:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is a popular evaluation metric for regression, which is defined as the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

2.5 Question 2.4 (Evaluate your predictions on test features with RMSE metric, 5 pts)

Hint: you may calculate MSE using scikit-learn, but RMSE is a little bit different from MSE.

[28]:
```
from sklearn import metrics
import math
################## Your answer for Question 2.4   ##################
mse = metrics.mean_squared_error(y_test, y_pred)
rmse = math.sqrt(mse)
# Print MSE and RMSE
print(f"{mse=}, {rmse=}")

################## Your code above ##################
```

```
mse=3.1740973539761033, rmse=1.78159966153345
```

2.6 Question 2.5 (Does the feature 'Newspaper' help? 10 pts)

Does the feature **Newspaper** contribute or impair the model prediction? Now repeat previous procedure:

- For features X, only select **TV** and **Radio**; response y is unchanged.
- Split the dataset as what you have done before.
- Fit/Train the model on the train data.
- Make predictions on the test data.
- Compute the RMSE of the predictions on the test data.
- **Show your observations!**

Reminder: if you use random_state while splitting the dataset, make sure the value of random_state is the same for the two splits. So that the test data only differs in the column of **Newspaper**.

[29]:
```
################## Your answer for Question 2.5   ##################
X_no_news = data[['TV', 'Radio']]
y_no_news = y  # already only has Sales, so no change.
X_train_no_news, X_test_no_news, y_train_no_news, y_test_no_news =␣
 ↪train_test_split(
  X_no_news, y_no_news, test_size=0.2, random_state=42
)
# Sanity check that values are the same for the two splits.
assert ((X_train_no_news['TV'] == X_train['TV']).all() and
 (X_train_no_news['Radio'] == X_train['Radio']).all() and
  (y_train == y_train_no_news).all())

# Fit the model
model_no_news = LinearRegression().fit(X_train_no_news, y_train_no_news)

# Make predictions on the test
y_pred_no_news = model_no_news.predict(X_test_no_news)
test_preds_no_news = model_no_news.score(X_test_no_news,y_test_no_news)
```

```
print(f"{test_preds_no_news=}")

# Compute RMSE
mse_no_news = metrics.mean_squared_error(y_test_no_news, y_pred_no_news)
rmse_no_news = math.sqrt(mse_no_news)
print(f"{rmse_no_news=}")

################## Your code above #################
```

```
test_preds_no_news=0.9005833101920356
rmse_no_news=1.7714254173033517
```

2.6.1 Your observations:

We see that the the score is slightly higher without Newspaper, at 0.9006 without Newspaper vs 0.8994 with Newspaper. The RMSE is also slightly less, at 1.7714 without vs 1.7816 with. This indicates that Newspaper was likely not helpful in explaining sales, as the fit is similar (actually slightly better) without newspaper.

3 Question 3 (PCA, 40 pts)

In this question, we will explore how PCA helps with reducing the dimensionality through scikit-learn. We will adopt a simple digits dataset contained in scikit-learn.

[30]:
```
# import libraries
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns;
sns.set()
```

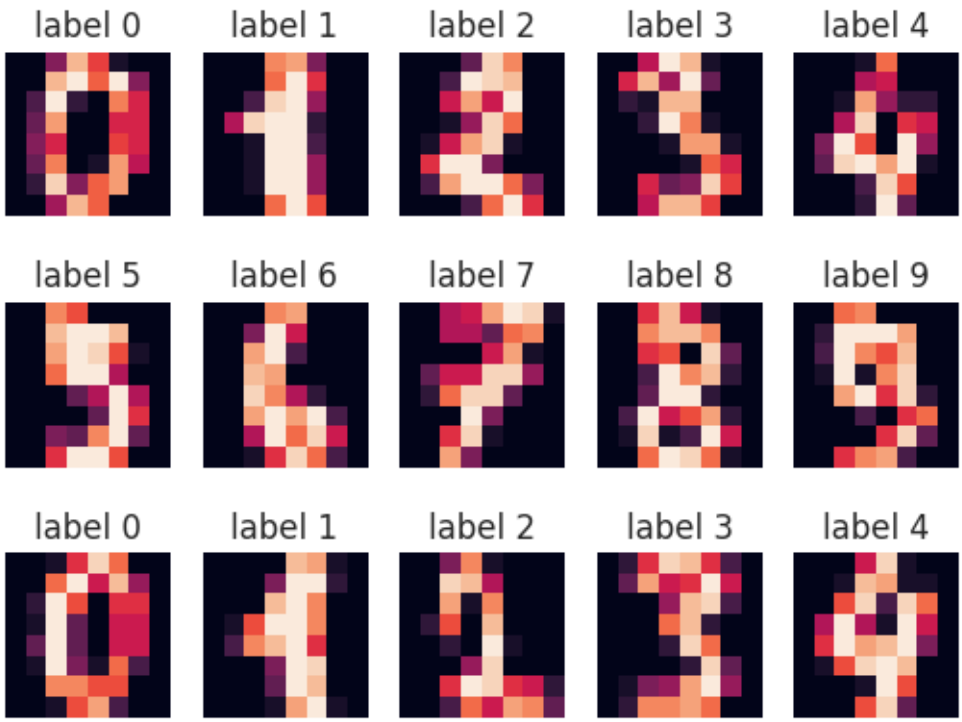
[31]:
```
from sklearn.datasets import load_digits
data_digit = load_digits()
data_digit.data.shape
```

[31]:
```
(1797, 64)
```

As you have seen above, there are 1797 rows and 64 columns in this dataset. Each row denotes the pixel values of an image. Since each image is 8*8 (in pixel), the dimension of each feature is 64. Let's take a look at a few images contained in this dataset.

[32]:
```
# A help function to display images with labels
def display_digits(imgs):
    h = 3
    w = 5
    fig, axes = plt.subplots(h, w)
    for i in range(h):
        for j in range(w):
            if imgs.shape[1] == 64:
```

```
                axes[i, j].imshow(imgs[i * w + j].reshape(8, 8))
            else:
                axes[i, j].imshow(imgs[i * w + j])
            axes[i, j].axis('off')
            axes[i, j].set_title(f'label {data_digit.target[i * w + j]}')
# Take a look at a few examples
display_digits(data_digit.images)
```



PCA finds uncorrelated orthogonal axes (principal components) in a high dimensional space (i.e., 64) to project the feature onto principal components.

3.1 Question 3.1 (PCA and dimensionality deduction with Scikit-learn, 10 pts)

In this question, you may use the implementation of PCA from scikit-learn, as imported below. Detailed explanations of parameters settings are available in this link.

3.1.1 Your task in this question:

Fit the model with feature X (**data_digit.data** for this dataset) and apply the dimensionality reduction on X, keep 2 components.

[33]:
```
from sklearn.decomposition import PCA

################## Your answer for Question 3.1   ##################
```

```
# name the transformed (dimension reduced) feature as projected
n_components = 2
pca = PCA(n_components=n_components)
projected = pca.fit_transform(data_digit.data)
################## Complete the code above #################

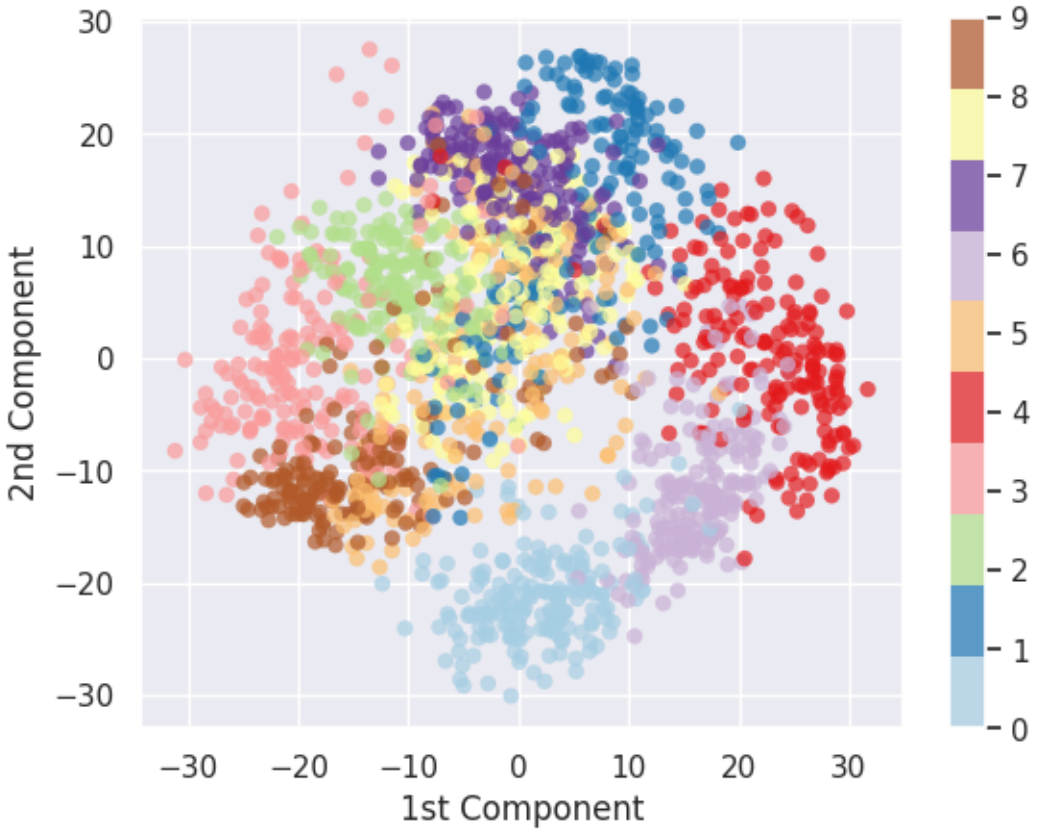

# The code below checks whether the feature dimension 64 is reduced to 2.
print(data_digit.data.shape)
print(projected.shape)
```

```
(1797, 64)
(1797, 2)
```

With the reduced dimension of features, we can now visualize the projected/reduced features in a 2-dimension plot as below. There are hardly any overlaps between digit 0 and any other digits. While in the central area of the figure, features are not well separable.

[34]:
```
plt.scatter(projected[:, 0], projected[:, 1],
            c=data_digit.target, edgecolor='none', alpha=0.7,
            cmap=plt.get_cmap('Paired', 10))
plt.xlabel('1st Component')
plt.ylabel('2nd Component')
plt.colorbar()
plt.figure(figsize=(18, 8))
```

[34]:
```
<Figure size 1800x800 with 0 Axes>
```




```
<Figure size 1800x800 with 0 Axes>
```

3.2 Question 3.2 (How many components do we need? 15 pts)

In practice, suppose you are given a high-dimension dataset, and you are interested in performing a dimension reduciton with PCA. How many conponents do we need? One method is to look at the distribution of the explained variance ratio w.r.t. each component.

In this question, you may use the function explained_variance_ratio in **pca** (with scikit-learn) to address this question. Detailed explanations of parameters settings are available in this link.

3.2.1 Your task in this question:

Print the ratio of variance explained by each of the selected components;

Visualize with matplotlib or seaborn (x axis: $i-th$ component; y axis: the corresponding ratio of explained variance).

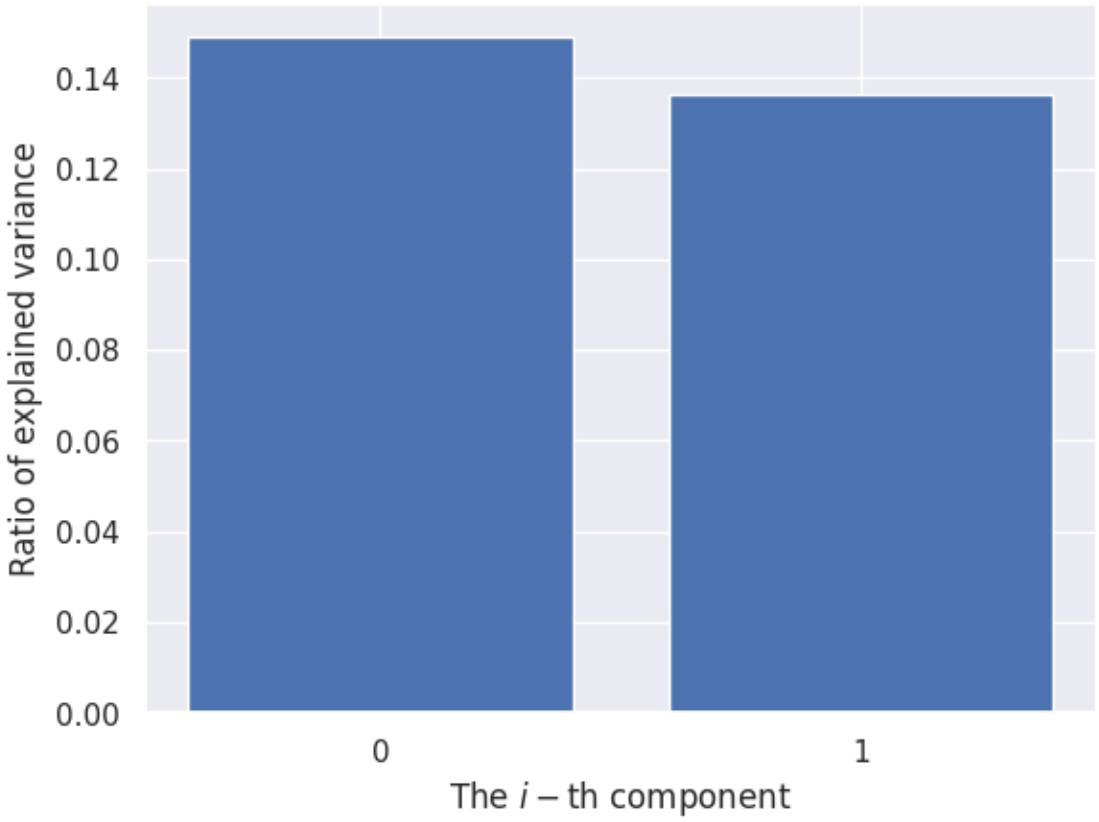
[35]:
```
################## Your answer for Question 3.2   ##################
explained_variance_ratio = pca.explained_variance_ratio_
print(f"{explained_variance_ratio=}")
```

```
# Plot x axis 0, 1; y axis corresponding ratios.
plt.bar([0, 1], explained_variance_ratio)
plt.xticks([0, 1])

plt.xlabel('The $i-$th component')
plt.ylabel('Ratio of explained variance')
################## Complete the code above #################
```

```
explained_variance_ratio=array([0.14890594, 0.13618771])
```

[35]:
```
Text(0, 0.5, 'Ratio of explained variance')
```



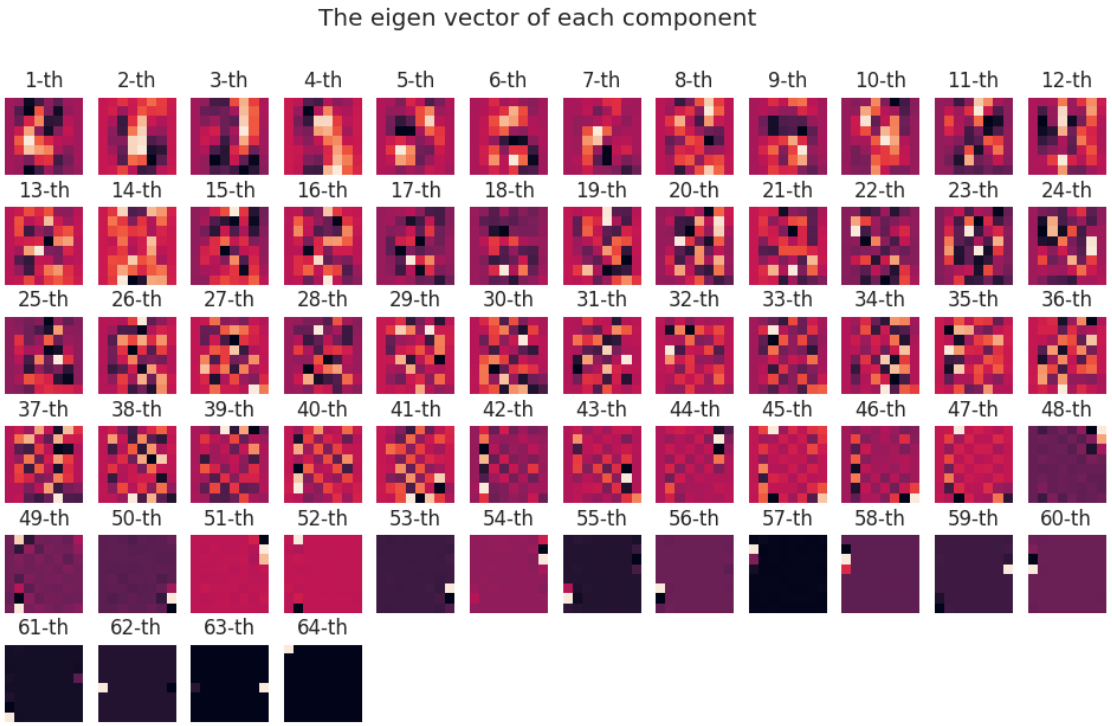
3.2.2 How to obtain eigen values and vectors from sklearn PCA

- Eigen values – your-PCA-model.explained_variance_
- Eigen vectors – your-PCA-model.components_

3.2.3 Eigen vectors (Eigen digits)

[36]:
```
# Default # of components is the dimension of feature space
pca = PCA()
pca.fit(data_digit.data)
h = 6
w = 12
fig, axes = plt.subplots(h, w)
fig.set_size_inches(12, 7)
for i in range(h):
    for j in range(w):
        idx = i * w + j
        if idx > 63:
            axes[i, j].axis('off')
        else:
            axes[i, j].imshow(pca.components_[idx].reshape(8, 8))
            axes[i, j].axis('off')
            axes[i, j].set_title(f'{idx + 1}-th')
fig.suptitle('The eigen vector of each component')
```

[36]:
```
Text(0.5, 0.98, 'The eigen vector of each component')
```



3.3 Question 3.3 What are your observations from the visualized eigen vectors? (5 pts)

3.3.1 Your solutions for Question 3.3:

The lower eigenvectors appear to be more shaped and more closely representing numbers; for example, the 1st eigenvector looks like it could represent a 6; the second, a 1. These seem to match more closely onto digits, or at least, parts of digits, that are used throughout the dataset. As the components corresponding to the eigenvectors increase, they appear to be less and less interpretable without any clear shape, perhaps more representing the noise than any structure. Then, the last quarter of the components have eigenvectors corresponding to almost entirely one color, perhaps representing the different backgrounds and subtle noise. This appears to align with our intuition, which is that the first components will account for most of the variance and thus be more representative of actual visual aspects that make up or differentiate instances in the dataset.

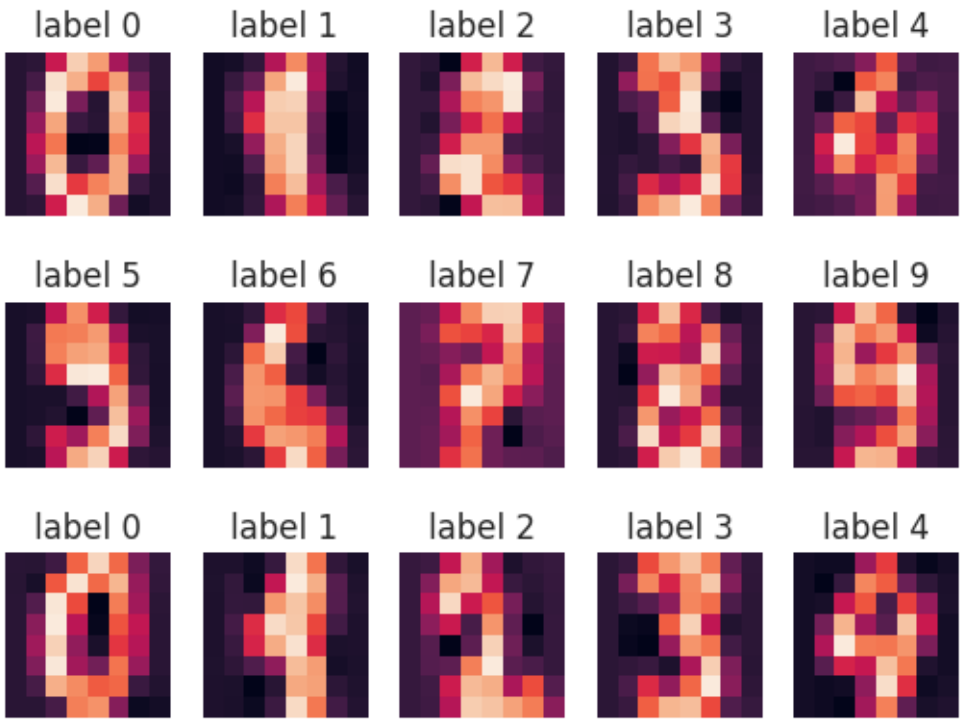
3.3.2 Feature reconstruction:

Suppose you only want to preserve 80% variance after applying PCA for the dimension reduction, due to storage issues, etc.

[37]:
```
# Use pca from sklearn to fit on the feature space
pca_08 = PCA(0.8).fit(data_digit.data)

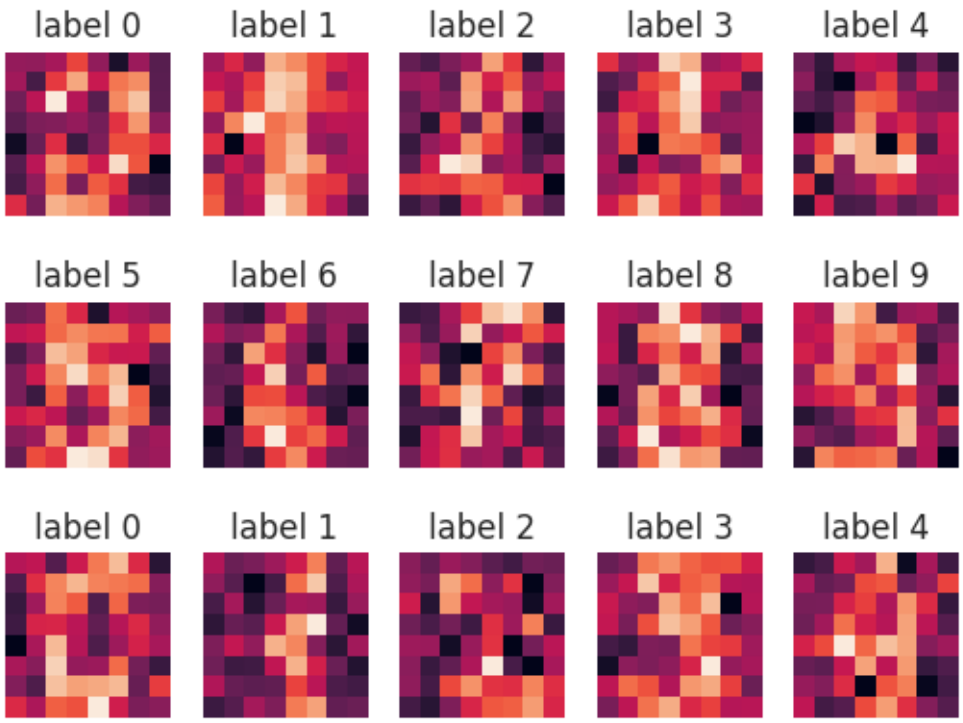
# Reconstruct
lower_dimension = pca_08.fit_transform(data_digit.data)
reconstructed = pca_08.inverse_transform(lower_dimension)

# Display the reconstructed images (with a much lower dimension)
display_digits(reconstructed)
```



Assume now you are given only a set of perturbed images (with some random noise). As shown below, the figures become much more difficult to recognize.

[38]:
```
# Apply random noise on clean images
noisy_imgs = np.random.normal(data_digit.data, 5)
# Display the noisy images
display_digits(noisy_imgs)
```



3.4 Question 3.4 Does PCA mitigate the random noise? (10 pts)

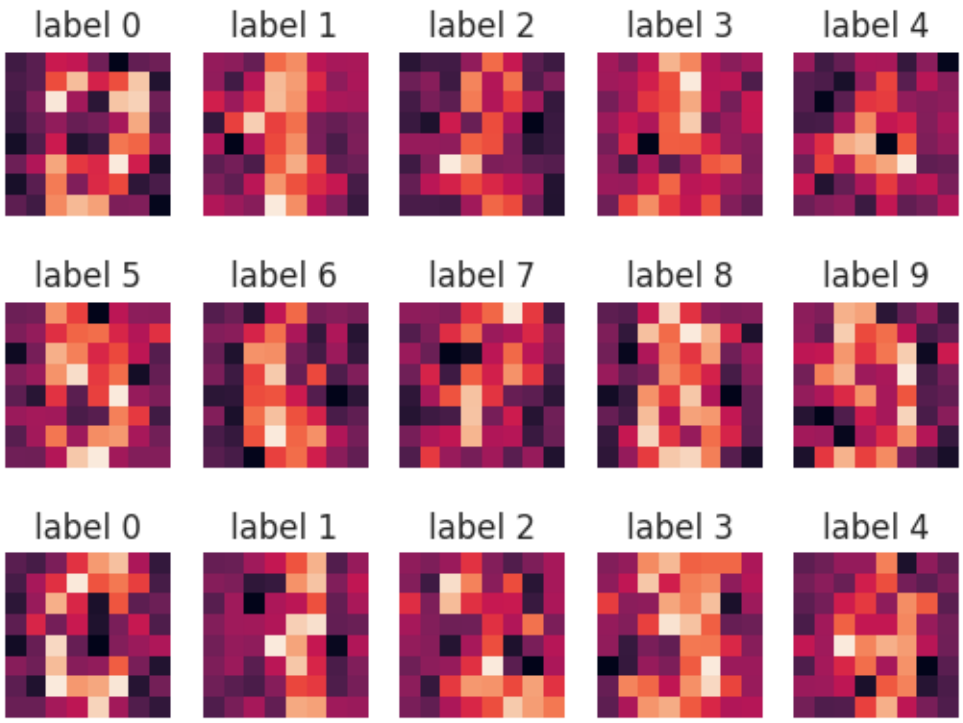
3.4.1 Your tasks:

- Perform feature reconstruction on noisy_imgs (preserving 80% variance after applying PCA for the dimension reduction)
- Explain your understandings on why PCA reconstructs cleaner images.

[39]:
```
################## Your answer for Question 3.4   ##################


pca_noisy = PCA(0.8).fit(noisy_imgs)
lower_dimension_noisy = pca_noisy.fit_transform(noisy_imgs)

# name the reconstructed images from noisy images as 'reconstructed_noise'
reconstructed_noise = pca_noisy.inverse_transform(lower_dimension_noisy)
# Display the reconstructed images
display_digits(reconstructed_noise)
################## Complete the code above #################
```



3.4.2 Your written answers for Question 3.4:

PCA constructs clearer images because it finds the most important parts of the imgs by finding the directions of most variation. When there's more noise, PCA will still try to find the fundamental parts of the images, and thus the reconstructions are kind of getting rid of the noise, since they are only taking the most important parts.